

# Documentazione Tecnica e Architetture di ComponentDB

La complessità emergente nella gestione dell'inventario hardware e della prototipazione elettronica richiede strumenti informatici in grado di coniugare un'alta reattività dell'interfaccia utente con un backend robusto e leggero. ComponentDB è un'applicazione web sviluppata appositamente per rispondere a questa esigenza, funzionando come un database locale per componenti elettronici. L'applicativo integra funzionalità avanzate di scansione di codici a barre, elaborazione dati e tracciamento storico, appoggiandosi su uno stack tecnologico moderno basato su Python e tecnologie web standard.

Questa documentazione esplora esaurientemente l'architettura del sistema, le scelte implementative, le dipendenze tecnologiche e le implicazioni ingegneristiche di secondo e terzo ordine derivanti dall'adozione di framework aggiornati. Nonostante la profondità dell'analisi, la struttura è progettata per risultare accessibile e di facile consultazione, nel rispetto dei requisiti di chiarezza e sintesi operativa.

## Architettura del Frontend e User Experience

L'interfaccia utente di ComponentDB è consolidata all'interno di un singolo file denominato `index.html`, il quale opera come una Single Page Application (SPA).<sup>1</sup> Il design è stato meticolosamente studiato per ridurre il carico cognitivo dell'operatore durante le sessioni intensive di magazzino, adottando un tema visivo scuro (dark mode) con accenti di colore cibernetici che facilitano la concentrazione visiva.

### Configurazione del Viewport e Tipografia

A livello di metadati HTML5, il documento definisce la lingua inglese (`<html lang="en">`) e la codifica caratteri UTF-8 (`<meta charset="UTF-8">`), garantendo la corretta visualizzazione di simboli tecnici.<sup>1</sup> Un elemento cruciale per l'usabilità su dispositivi mobili è la configurazione del viewport: `<meta name="viewport" content="width=device-width, initial-scale=1.0, maximum-scale=1.0, user-scalable=no">`.<sup>1</sup> Il blocco della scalabilità (`user-scalable=no`) previene ingrandimenti accidentali dello schermo durante l'interazione touch o la scansione, conferendo alla pagina web un comportamento responsivo del tutto assimilabile a quello di un'applicazione mobile nativa.

Il sistema tipografico si affida a Google Fonts, importando due famiglie di caratteri distintive:

- JetBrains Mono: Un font monospazio ottimizzato per lo sviluppo software, utilizzato per garantire l'allineamento perfetto dei codici prodotto (Part Numbers) e dei dati numerici.<sup>1</sup>
- Rajdhani: Un font dal design moderno e squadrato, impiegato per le intestazioni e il branding (logo), che rafforza l'estetica tecnologica dell'applicazione.<sup>1</sup>

### Librerie Esterne e Motore di Scansione

Per minimizzare il peso del codice sorgente e massimizzare l'affidabilità, il frontend delega funzionalità complesse a librerie esterne consolidate:

- **html5-qrcode (v2.3.8)**: Importata tramite tag `<script>`, questa libreria gestisce l'accesso alla fotocamera del dispositivo sfruttando le API WebRTC, occupandosi del parsing dei frame video in tempo reale per l'identificazione di codici QR e codici a barre lineari.<sup>1</sup>
- **xlsx.full.min.js (v0.18.5)**: Caricata via CDN, abilita il browser all'elaborazione e all'esportazione di fogli di calcolo Excel, trasferendo il carico computazionale dell'elaborazione dati dal server al client dell'utente.<sup>1</sup>

## Sistema di Design e Variabili CSS

La coerenza visiva è mantenuta attraverso un uso intensivo di variabili CSS (Custom Properties) definite nella root del documento (`:root`). Questo approccio centralizzato facilita enormemente eventuali implementazioni future di temi aggiuntivi. La seguente tabella illustra la gerarchia cromatica e funzionale del sistema:

| Categoria Variabili | Variabile CSS             | Valore Esadecimale        | Scopo Architettuale                                                                    |
|---------------------|---------------------------|---------------------------|----------------------------------------------------------------------------------------|
| Sfondi (Background) | --bg, --bg1, --bg2, --bg3 | #080c10 -> #21262d        | Creare profondità tramite una scala di grigi scuri per pannelli e schede. <sup>1</sup> |
| Bordi (Borders)     | --border, --border2       | #30363d, #3d444d          | Delimitare contenitori e card mantenendo basso contrasto. <sup>1</sup>                 |
| Testo (Text)        | --text, --muted           | #c9d1d9, #768390          | Fornire leggibilità prolungata ed evidenziare informazioni secondarie. <sup>1</sup>    |
| Accenti (Accents)   | --cyan, --cyan-dim        | #00d4ff, #00a8cc          | Richiamare azioni primarie e contorni interattivi. <sup>1</sup>                        |
| Stati (Status)      | --green, --red, --amber   | #3fb950, #f85149, #d29922 | Identificare la disponibilità di magazzino (es. ok,                                    |

|                          |                             |                           |                                                                                          |
|--------------------------|-----------------------------|---------------------------|------------------------------------------------------------------------------------------|
|                          |                             |                           | esaurito, in esaurimento). <sup>1</sup>                                                  |
| <b>Fornitori (Brand)</b> | --lcsc, --digikey, --mouser | #e36209, #cc0000, #007ac2 | Riconoscimento pre-attentivo istantaneo del fornitore tramite badge visivi. <sup>1</sup> |

L'interfaccia si sviluppa attorno a un layout dinamico. L'intestazione superiore (header) ospita il logo e una barra statistica dotata di indicatori di stato animati con effetto pulsazione (@keyframes pulse).<sup>1</sup> La navigazione a schede altera le regole CSS display dei vari pannelli (#panels), nascondendo e mostrando i contenuti tramite l'applicazione della classe .active.<sup>1</sup> Il pannello di scansione (#panel-scanner) adotta una griglia adattiva: su display ampi utilizza una vista divisa con una barra laterale fissa di 380px per i risultati, mentre su schermi con larghezza inferiore a 900px collassa in un layout a singola colonna.<sup>1</sup> L'esperienza visiva è arricchita da un overlay della fotocamera (.scan-overlay) che proietta parentesi angolari e una linea di scansione animata che cicla ogni 2 secondi (--scan-speed: 2s).<sup>1</sup>

## Architettura del Backend e Transizione a Flask 3.0.0

Il motore logico dell'applicazione risiede nello script Python app.py, il quale agisce da server web RESTful.<sup>1</sup> Il sistema è stato ingegnerizzato dichiarando una dipendenza formale stringente nel file requirements.txt: flask>=3.0.0.<sup>1</sup>

Questa specifica dipendenza non è arbitraria, ma porta con sé implicazioni architetturali profonde. Rilasciato storicamente nel settembre 2023<sup>2</sup>, Flask 3.0.0 rappresenta un aggiornamento maggiore che impone l'adozione di Werkzeug 3.0.0 o superiore.<sup>2</sup> La ristrutturazione del codice base di Flask ha introdotto le basi "Sans-IO" per le classi principali Flask e Blueprint.<sup>2</sup> L'architettura Sans-IO separa la logica di parsing del protocollo dalla gestione concreta degli stream di rete (I/O), rendendo il framework molto più testabile e predisposto ad architetture asincrone.

Un'ulteriore modifica impattante adottata in ComponentDB riguarda la serializzazione dei dati. In Flask 3.0.0, l'istanza flask.json è divenuta un provider dedicato (JSONProvider) configurabile dinamicamente; le funzioni JSON interne non richiedono più il passaggio esplicito del parametro di codifica, in quanto gestite nativamente a livello di applicazione.<sup>2</sup> Inoltre, per innalzare gli standard di sicurezza sui percorsi del file system, è stata rimossa definitivamente la funzione deprecata safe\_join.<sup>2</sup> L'assenza di dipendenze legacy instabili (come le vecchie versioni di Flask-Login, le quali si sono rotte al lancio di Werkzeug 3.0<sup>3</sup>) dimostra che ComponentDB è stato concepito per operare in un ecosistema contemporaneo, pulito e resiliente.

## Strato di Persistenza e SQLite WAL

Il salvataggio dei dati è demandato a SQLite3, interfacciato nativamente tramite le librerie

standard di Python.<sup>1</sup> La scelta di SQLite, in contrapposizione a DBMS esterni come PostgreSQL o MySQL, azzerla la necessità di manutenzione del database, incapsulando l'intero inventario nel singolo file `components.db` localizzato nella directory dell'applicativo (`BASE_DIR`).<sup>1</sup> Per ovviare alle limitazioni storiche di SQLite in contesti multi-utente (come l'errore `database is locked`), la funzione `get_db()` invoca l'istruzione `PRAGMA journal_mode=WAL`.<sup>1</sup> La modalità Write-Ahead Logging altera radicalmente l'operatività del disco: le scritture vengono accodate in un file di log separato, consentendo letture e scritture simultanee. Se un operatore sta consultando l'inventario per visualizzare le statistiche e un altro sta scansionando dozzine di componenti in rapida sequenza, il sistema WAL assicura un'esecuzione concorrente fluida.<sup>1</sup> Le risposte del database vengono inoltre mappate dinamicamente come dizionari impostando il `row_factory` su `sqlite3.Row`.<sup>1</sup>

## Schema Dati e Integrità Relazionale

L'inizializzazione del database (`init_db()`) con `PRAGMA foreign_keys=ON` costruisce due entità strettamente accoppiate<sup>1</sup>:

### 1. Tabella `components`:

- Costituisce l'anagrafica. Presenta campi come `id` (Primary Key), `part_number` (Unique, Not Null), `manufacturer_part`, `description`, `supplier`, `quantity`, `package`, `value`, `category`, `datasheet_url` e `notes`.<sup>1</sup>
- Implementa il tracciamento temporale tramite `created_at` e `updated_at`, gestiti in tempo locale.<sup>1</sup>

### 2. Tabella `transactions`:

- Funge da registro immutabile degli eventi (Event Sourcing log).
- Collega il movimento all'anagrafica tramite `component_id` (Foreign Key).<sup>1</sup>
- Registra la cronologia matematica inequivocabile salvando `quantity_change`, `quantity_before` e `quantity_after`, oltre a specificare l'`operation`, la `source` (ad esempio, 'scan') e includendo il `barcode_raw` per auditing o decodifiche postume in caso di errori dell'algoritmo di lettura.<sup>1</sup>

Le query di estrazione sono ottimizzate dalla presenza di quattro indici B-Tree: `idx_comp_pn`, `idx_comp_supplier`, `idx_tx_component` e `idx_tx_timestamp`, garantendo una complessità temporale di interrogazione logaritmica.<sup>1</sup>

## Euristiche di Decodifica dei Codici a Barre

Il nucleo algoritmico del processing dei dati è racchiuso nelle funzioni `parse_lcsc_js_object` e `parse_barcode` in `app.py`.<sup>1</sup> La normalizzazione delle etichette, soprattutto quelle di componentistica di provenienza asiatica, presenta sfide non triviali. Le buste di componenti del fornitore LCSC, ad esempio, generano frequentemente codici a barre formattati come oggetti JavaScript privi di virgolette (es. `{pbn:PICK...,pc:C492427,pm:PZ...}`).<sup>1</sup> Poiché un analizzatore JSON standard fallirebbe su queste stringhe, la funzione dedicata elabora il testo rimuovendo le parentesi graffe e partizionando dinamicamente le chiavi identificando le virgole seguite da tag noti, estraendo così un dizionario Python valido.<sup>1</sup>

La funzione madre `parse_barcode` implementa una catena di responsabilità logica per assegnare parametri vitali (come `supplier`, `part_number`, e `quantity`) corredati da un punteggio di affidabilità (`confidence`) <sup>1</sup>:

- Tenta il parsing JSON/JS e cerca identificatori (prefissati dalla lettera "C") nei campi associati a part number (pc, pm, mpn).<sup>1</sup>
- Verifica tramite espressioni regolari (RegEx) se il formato consiste unicamente in un codice diretto (lettera "C" seguita da 4 a 8 cifre).<sup>1</sup>
- Analizza layout multilinea: se la prima riga è un Part Number valido, deduce che la seconda possa rappresentare la quantità intera.<sup>1</sup>
- In caso di totale insuccesso euristico, preserva l'integrità del dato in un formato di fallback: il supplier diventa "Unknown", il sistema assorbe i primi 120 caratteri grezzi, e la bassa confidenza allerta l'utente via interfaccia per favorire un intervento umano mirato.<sup>1</sup>

## Interfaccia API e Logica Transazionale

Il backend espone servizi CRUD (Create, Read, Update, Delete) mappati su specifici percorsi REST, con logiche di mitigazione degli errori e gestione della sicurezza dei dati.<sup>1</sup>

| Metodo | Endpoint HTTP   | Funzionalità e Implicazioni Logiche                                                                                                                                                                                                                                                                              |
|--------|-----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| POST   | /api/scan       | Accetta il payload dal client. Applica un "Upsert": se il componente esiste, aggiorna i dati vacanti e ricalcola la quantità limitando i valori sotto lo zero ( $\max(0, \text{qty\_before} + \text{qty\_change})$ ). Se nuovo, inserisce il record. Crea sempre un log di transazione contestuale. <sup>1</sup> |
| GET    | /api/components | Ricerca (parametro query q insensibile a maiuscole), ordinamento tramite allow-list rigorosa per evitare SQL Injection, e limite di paginazione cautelativo a 1000 righe per preservare l'uso della RAM                                                                                                          |

|        |                       |                                                                                                                                                                                                                                                       |
|--------|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|        |                       | lato client. <sup>1</sup>                                                                                                                                                                                                                             |
| PUT    | /api/components/<cid> | Permette modifiche dirette ai metadati (descrizioni, link datasheet). Se viene rettificata forzatamente la quantità, il sistema genera un record di transazione manuale compensativo in automatico per quadrare i bilanci logici. <sup>1</sup>        |
| DELETE | /api/components/<cid> | Esegue un "cascade" applicativo: rimuove prioritariamente le righe in transactions prima del componente anagrafico padre, tutelando i vincoli di foreign key. <sup>1</sup>                                                                            |
| POST   | /api/query            | Accesso SQL crudo per diagnostica interna. Di base confinato unicamente ad operazioni testuali di tipo SELECT (limite 2000 tuple). Impone l'esplicito passaggio del flag allow_write per sbloccare istruzioni manipolative distruttive. <sup>1</sup>  |
| GET    | /api/stats            | Aggrega indicatori di performance (KPI): conteggio componenti unici, volume globale, articoli sottoscorta (quantità fra 0 e 5) o esauriti (0), frequenza di aggiornamento sulle ultime 24 ore e ripartizione patrimoniale per fornitore. <sup>1</sup> |

## Sicurezza, Crittografia e Provisioning di Rete

Uno dei limiti infrastrutturali per lo sviluppo web locale risiede nei protocolli di sicurezza dei

browser moderni: l'accesso all'hardware video (getUserMedia per le API WebRTC) su client remoti collegati alla stessa LAN è bloccato attivamente se servito tramite connessioni non cifrate (HTTP), ad eccezione del solo localhost.<sup>1</sup>

Per aggirare questo scoglio e consentire l'utilizzo immediato via tablet o smartphone sulla rete WiFi aziendale, app.py provvede alla creazione istantanea (on-the-fly) di un certificato SSL autofirmato invocando generate\_ssl\_cert() in assenza dei file cert.pem e key.pem.<sup>1</sup>

Questo meccanismo poggia saldamente sulla dipendenza indicata nei requisiti: cryptography>=42.0.0.<sup>1</sup> L'uso della release 42.0.0 (pubblicata a gennaio 2024)<sup>6</sup> introduce miglorie di sicurezza eccezionali. Tale versione rimuove il supporto per LibreSSL in versioni precedenti alla 3.7<sup>7</sup>, appoggiandosi per contro sulle precompilazioni binarie fornite internamente basate sull'implementazione standard OpenSSL 3.2.<sup>7</sup> Grazie a questa evoluzione, l'infrastruttura supporta funzioni matematiche moderne come l'RSA PSS (Probabilistic Signature Scheme) negli oggetti PKCS7 e X.509<sup>7</sup>, e suite crittografiche come ChaCha20, SM4 GCM e AESGCMsiv, innalzando i livelli difensivi rispetto ai generatori di chiavi legacy.<sup>7</sup> Il certificato, compilato con una chiave RSA a 2048-bit, viene fornito di una validità estesa di 3650 giorni (10 anni) per annullare l'onere amministrativo del rinnovo in rete locale.<sup>1</sup> Inoltre, il caricamento di documenti con seriali negativi (X.509) o contenuti assenti lancerà un'eccezione ferrea ValueError<sup>7</sup>, garantendo il fallimento esplicito su chiavi corrotte.

Al fine di prevenire gli errori browser di "Invalid Name", il sistema rileva gli indirizzi di rete per compilare correttamente il campo Subject Alternative Name (SAN).<sup>1</sup> Lo script esegue uno scan locale IPv4/IPv6 tramite get\_all\_local\_ips() ed esegue un test empirico con get\_local\_ip(), aprendo una transazione UDP (Connectionless) rapida verso il DNS Google 8.8.8.8 sulla porta 80 per scoprire quale interfaccia di routing viene utilizzata attivamente per il traffico uscente, ricorrendo al fallback su 127.0.0.1 solo in assenza di percorsi attivi.<sup>1</sup> Durante la fase di avvio del main block di Flask, il framework espone infine un banner interattivo su console con l'URL corretto da digitare nei dispositivi mobili per l'accesso HTTPS sulla porta 5000 (host="0.0.0.0", threaded=True).<sup>1</sup>

## Ambienti Virtuali e Procedure di Distribuzione

L'esecuzione del pacchetto, a fronte della potenziale conflittualità fra i requisiti di sistema e le dipendenze complesse come cryptography 42.0.0 (il cui albero di dipendenze può entrare in collisione con pacchetti paralleli globali come snowflake-connector-python<sup>9</sup>), impone metodologicamente la segregazione all'interno di un ambiente virtuale (Virtual Environment). Il modulo standard Python venv fornisce ambienti isolati a basso costo prestazionale.<sup>10</sup>

Le istruzioni formali di attivazione per istanziare ComponentDB sulle varie infrastrutture informatiche sono riassunte di seguito:

| Piattaforma | Interprete<br>Comandi (Shell) | Comando<br>Generazione | Comando<br>Attivazione |
|-------------|-------------------------------|------------------------|------------------------|
|-------------|-------------------------------|------------------------|------------------------|

|                              |                             |                                       |                                             |
|------------------------------|-----------------------------|---------------------------------------|---------------------------------------------|
| <b>Linux / macOS / POSIX</b> | Bash / Zsh                  | python3 -m<br>venv.venv <sup>10</sup> | source.venv/bin/activate <sup>10</sup>      |
| <b>Linux / macOS / POSIX</b> | Fish Shell                  | python3 -m<br>venv.venv <sup>12</sup> | source.venv/bin/activate.fish <sup>12</sup> |
| <b>Linux / macOS / POSIX</b> | Csh / Tcsh                  | python3 -m<br>venv.venv <sup>12</sup> | source.venv/bin/activate.csh <sup>12</sup>  |
| <b>Windows</b>               | Prompt Comandi<br>(cmd.exe) | python -m<br>venv.venv <sup>10</sup>  | .venv\Scripts\activate.bat <sup>10</sup>    |
| <b>Windows</b>               | PowerShell                  | python -m<br>venv.venv <sup>12</sup>  | .venv\Scripts\Activate.ps1 <sup>12</sup>    |

Per abbandonare l'ambiente virtuale una volta terminata l'operatività del server Flask, è sufficiente digitare l'istruzione universale deactivate all'interno del terminale, ripristinando il path globale del sistema ospitante.<sup>12</sup>

L'applicazione, combinando l'autosufficienza della base dati relazionale locale, la decodifica adattiva di standard logistici non ortodossi, e il consolidamento della sicurezza crittografica per lo sblocco nativo dell'acquisizione hardware locale, risulta una piattaforma integrata che accelera radicalmente l'operatività d'officina, garantendo stabilità a lungo termine e flessibilità di indagine tramite architetture di rete e web resilienti.

## Bibliografia

1. index.html
2. Changes — Flask Documentation (3.1.x), accesso eseguito il giorno maggio 28, 2026, <https://flask.palletsprojects.com/en/stable/changes/>
3. We Have To Talk About Flask : r/Python - Reddit, accesso eseguito il giorno maggio 28, 2026, [https://www.reddit.com/r/Python/comments/17bnmxx/we\\_have\\_to\\_talk\\_about\\_flask/](https://www.reddit.com/r/Python/comments/17bnmxx/we_have_to_talk_about_flask/)
4. Flask 3.0.0 Released : r/Python - Reddit, accesso eseguito il giorno maggio 28, 2026, [https://www.reddit.com/r/Python/comments/16w97i6/flask\\_300\\_released/](https://www.reddit.com/r/Python/comments/16w97i6/flask_300_released/)
5. What's New In Flask 3.0 - DEV Community, accesso eseguito il giorno maggio 28, 2026, <https://dev.to/khaleelhabeeb/whats-new-in-flask-30-2g5a>
6. cryptography 42.0.0 - PyPI, accesso eseguito il giorno maggio 28, 2026, <https://pypi.org/project/cryptography/42.0.0/>
7. Changelog — Cryptography 42.0.0 documentation, accesso eseguito il giorno maggio 28, 2026, <https://cryptography.io/en/42.0.0/changelog/>
8. [Python-announce] PyCA cryptography 42.0.0 released - Google Groups, accesso eseguito il giorno maggio 28, 2026,

- <https://groups.google.com/g/comp.lang.python.announce/c/9b-NNaJoFd4>
9. SNOW-1017272: Cannot install snowflake-connector-python with cryptography>=42.0.0 · Issue #1862 - GitHub, accesso eseguito il giorno maggio 28, 2026,  
<https://github.com/snowflakedb/snowflake-connector-python/issues/1862>
  10. Python Virtual Environments: A Primer, accesso eseguito il giorno maggio 28, 2026, <https://realpython.com/python-virtual-environments-a-primer/>
  11. Python virtual environment / venv | Windows, Linux & Mac OS X - YouTube, accesso eseguito il giorno maggio 28, 2026,  
<https://www.youtube.com/watch?v=aoEMJJLUdJM>
  12. How can I create a virtual environment with Python 3? - Stack Overflow, accesso eseguito il giorno maggio 28, 2026,  
<https://stackoverflow.com/questions/43069780/how-can-i-create-a-virtual-environment-with-python-3>
  13. venv — Creation of virtual environments — Python 3.14.5 documentation, accesso eseguito il giorno maggio 28, 2026,  
<https://docs.python.org/3/library/venv.html>
  14. Creating a Python virtual environment - Advanced Research Computing at Hopkins, accesso eseguito il giorno maggio 28, 2026,  
<https://www.arch.jhu.edu/python-virtual-environments/>